

Programmazione e calcolo scientifico

Esercitazione 9 - BFS, DFS e Dijkstra

12 maggio 2026

1 Obiettivo dell'esercitazione

Utilizzando la struttura dati grafo implementata nell'esercitazione precedente, si implementino gli algoritmi BFS, DFS e Dijkstra per la visita dei grafi.

2 Funzioni template

Una *funzione template* in C++ è una funzione che può operare su tipi generici. Si consideri per esempio la seguente funzione che calcola il massimo tra due elementi:

```
template<typename T>
T max(const T& a, const T& b)
{
    if (a < b)
        return b;
    else
        return a;
}
```

Si legga `template<typename T>` come se fosse un $\forall T$. Fintantoché il tipo `T` mette a disposizione un operatore di confronto, ed in particolare l'operatore di "minore", la definizione è valida. Generalmente il compilatore è in grado di dedurre `T` dal tipo **degli argomenti**, come nelle seguenti chiamate:

```
double d = max(1.0, 2.0); // dedotto double perché gli argomenti sono double
int i = max(1, 2); // dedotto int perché gli argomenti sono int
auto d2 = max(1.0, 2.0); // di nuovo double
std::string str1 = "alpha";
std::string str2 = "bravo";
auto s = max(str1, str2); // < su std::string confronta lessicograficamente
```

3 Modalità di implementazione di BFS e DFS

3.1 Prerequisiti

È stato visto a lezione che la visita in ampiezza e la visita in profondità di un grafo si effettuano appoggiandosi rispettivamente alle strutture dati “coda” e “stack”, che sono contenitori con politiche rispettivamente FIFO (First In First Out) e LIFO (Last In First Out). Si implementino quindi due classi “contenitore”¹ aventi entrambe i seguenti metodi:

- costruttore di default,
- metodo `put()` per inserire un elemento nel contenitore,
- metodo `get()` per rimuovere un elemento dal contenitore,
- metodo `empty()` per determinare se il contenitore è vuoto.

Le due classi devono fornire rispettivamente un contenitore con politica FIFO (First In First Out, coda) e uno con politica LIFO (Last In First Out, stack), come da Figura 1. Si implementi le due classi **incapsulando un opportuno contenitore della STL** e inoltrando le chiamate ai metodi `put()`, `get()` ed `empty()` verso le opportune operazioni del contenitore incapsulato.

Nota: le classi richieste si implementano in meno di 10 righe ciascuna. Fare attenzione a non reinventare la ruota, non è richiesto di implementare le strutture dati ma di adattare gli opportuni contenitori già presenti nella STL.

```
1  /* Contenitore con politica FIFO */
2  fifo q;
3  q.put(10);           // inserisce un elemento in fondo alla coda
4  int x = q.get();     // estrae l'elemento in testa alla coda
5  if ( q.empty() ) {  // verifica se il contenitore è vuoto
6      ...
7  }
8
9  /* Contenitore con politica LIFO */
10 lifo l;
11 l.put(10);           // inserisce un elemento in cima allo stack
12 int x = l.get();     // estrae l'elemento sulla cima dello stack
13 if ( l.empty() ) {  // verifica se il contenitore è vuoto
14     ...
15 }
```

Figura 1: Esempio dell’uso atteso delle classi implementate

¹Il termine “contenitore” qui è usato in modo generale, non con il significato specifico alla STL.

3.2 Implementazione della visita in ampiezza e profondità

Si implementi una singola funzione template `graph_visit()` che:

- Prende in input due parametri, il grafo da visitare ed il nodo sorgente
- Prende un parametro template che specifica il tipo del contenitore da usare per effettuare la visita
- Restituisce un oggetto di tipo grafo contenente l'albero risultante dalla visita

La funzione, in base al contenitore passato come parametro, deve effettuare una visita in ampiezza oppure una visita in profondità (Figura 2). Si ottengano le immagini dei due grafi utilizzando GraphViz e si verifichi che è stata effettuata la visita richiesta.

```
1  /* Effettua una visita in profondità di G, restituendo il risultato in dfsG */
2  lifo s;
3  auto dfsG = graph_visit(G, 1, s);
4  /* Effettua una visita in ampiezza di G, restituendo il risultato in bfsG */
5  fifo q;
6  auto bfsG = graph_visit(G, 1, q);
```

Figura 2: Esempio di chiamate alla funzione `graph_visit()`.

Si implementi una funzione `recursive_dfs()` che

- Prende in input due parametri, il grafo da visitare ed il nodo sorgente
- Restituisce un oggetto di tipo grafo contenente l'albero risultante dalla visita

e che usa la versione ricorsiva della DFS per visitare il grafo.

3.3 Algoritmo di Dijkstra

Si implementi l'algoritmo di Dijkstra utilizzando la struttura dati grafo dell'esercitazione precedente.

4 Approfondimento

Le due classi che realizzano i contenitori LIFO e FIFO di cui è richiesta l'implementazione nella Sezione 3.1 realizzano quello che è un *design pattern* conosciuto con il nome di *adapter*.

I design pattern sono soluzioni “standardizzate” a problemi ricorrenti nella programmazione: in questo caso si vuole adattare l'interfaccia di uno o più container della STL all'interfaccia attesa dalla funzione `graph_visit()`, cioè i metodi `put()`, `get()` ed `empty()`.

Nel libro *Design Patterns: Elements of Reusable Object-Oriented Software* del 1994 sono stati codificati 23 design patterns - tra cui l'*adapter* - tuttora molto utilizzati. Si tratta un libro di enorme successo e una lettura fondamentale per approfondire il tema della progettazione orientata agli oggetti. I suoi quattro autori sono collettivamente conosciuti sotto il nome *Gang Of Four*.

5 Modalità di svolgimento e consegna

Deve essere consegnato un programma che implementi gli algoritmi descritti sopra più un `CMakeLists.txt` che lo compili. Il tutto deve essere posizionato in una cartella chiamata **esercitazione9** nel proprio repository.

La consegna delle esercitazioni è obbligatoria e da effettuarsi entro i termini prescritti. Il tempo assegnato per lo svolgimento di questa esercitazione è di una settimana. Termine ultimo per la consegna di questa esercitazione sono le ore 23.59 di Mercoledì 20/05/2025: fa fede il timestamp del commit taggato **consegna_es9** sul proprio repository. Eventuali giustificazioni vanno presentate a matteo.cicuttin@polito.it entro il termine ultimo previsto per la consegna dell'esercitazione.

Si ricordi che l'uso di strumenti di intelligenza artificiale è scoraggiato ed in ogni caso dev'essere dichiarato esplicitamente all'interno dei sorgenti consegnati.

All'esame potrebbe essere richiesto di mostrare il funzionamento del codice, pertanto ci si assicuri che il codice nel proprio repository sia funzionante.